

DEEPL - TP 1

Function definition in pyTorch

Nicolas Baskiotis - Stéphane Rivaud - Benjamin Piwowarski - Laure Soulier

2025-2026

Important

On your *own computer*, you can reproduce the working environment by following the instructions at <https://mind.sorbonne-universite.fr/informations/logiciel/>. On the computers in the practical session rooms, this environment is already present.

```
source /users/nfs/Enseignants/piwowarski/venv/deepdac/bin/activate
```

If you use an editor (VS Code), don't forget to specify the environment to facilitate development.

Notations

x	a scalar
$\mathbf{x}$	a vector in $\mathbb{R}^n$
$\mathbf{X}$	a matrix in $\mathbb{R}^{n \times p}$
$\frac{\partial f}{\partial x}$	The partial derivative of f with respect to x .
$\frac{\partial f}{\partial \mathbf{x}}$ or $\nabla_{\mathbf{x}} f$	Vector of partial derivatives of f with respect to $\mathbf{x}_i$.
$\frac{\partial f}{\partial \mathbf{X}}$	Vector of partial derivatives of f with respect to $\mathbf{X}_{ij}$.

1 Function Graphs

A central element of **PyTorch** is the computation graph : when computing a variable, all the operations that were used in the computation are stored in the form of a computation graph. This graph is *acyclic*. The internal nodes of the graph represent the operations, the terminal node the result and the roots the input variables. This graph is used in particular to compute the partial derivatives of the output with respect to the input variables – using the chain rule of composed functions.

Function Graph In these practical sessions, we will consider that any neural network can be expressed as a composition of basic functions (transformation – e.g. linear ; activation – e.g. sigmoid, softmax ; error – e.g. quadratic error).

An example is given in figure 1 – for now, we are not interested in the nature of this graph, but more in its formalization : we have an input ($\mathbf{x}$), parameters (θ) and three functions (linear, transpose, quadratic error). The output is a scalar $\ell \in \mathbb{R}$. The computation graph performs the following operations : $\mathbf{y} = \theta\mathbf{x} + b_1$, $\hat{\mathbf{x}} = \theta^\top \mathbf{y} + b_2$, $\ell = \Delta(\mathbf{x}, \hat{\mathbf{x}})$ which computes a cost.

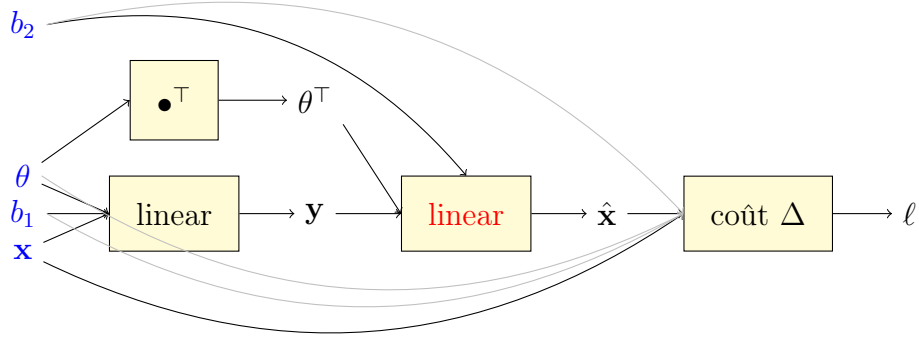


FIGURE 1 – A computation graph (auto-encoder)

The objective function that gives the output ℓ is $L(\mathbf{x}, b_1, \theta, b_2)$ – it therefore depends on four tensors (here scalars, vectors and matrices). To learn, we need to be able to compute the gradient of the risk ∇_L with respect to the parameters (here θ , b_1 and b_2).

Function To abstract this computation graph, we focus on the linear function in red in figure 1 : we place ourselves in the general framework where we consider all the operations that occur downstream as being produced by a function f and all those upstream by a function L as illustrated by figure 2.

We therefore have the following notation correspondences for figure 2 :

$\mathbf{m}$ corresponds to the initial inputs $(b_2, \theta, b_1, \mathbf{x})$
 $\mathbf{n}$ corresponds to the inputs of **linear** $(b_2, \theta^\top, \mathbf{y})$
 $\mathbf{z}$ corresponds to $\hat{\mathbf{x}}$
 f corresponds to $(b_2, \theta, b_1, \mathbf{x}) \mapsto (b_2, \theta^\top, \mathbf{y})$, i.e. $\mathbf{n} = f(\mathbf{m})$
 i.e. all transformations that precede the **linear** function
 g corresponds to **linear**, i.e. $\mathbf{z} = g(\mathbf{n})$
 L corresponds to cost, i.e. $\ell = L(\mathbf{m}, \mathbf{z})$

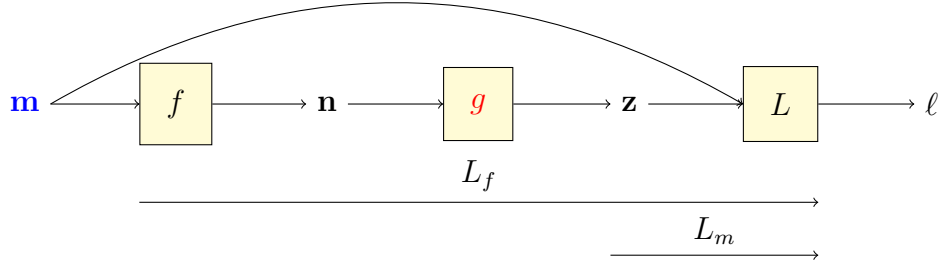


FIGURE 2 – A function g in the computation graph. f and L represent all the functions that come before/after in the computation graph.

To understand the backpropagation mechanism in practice, we focus on the function g (figure 2). The inputs $\mathbf{m}$ correspond to observations and parameters, and $\mathbf{n} = f(\mathbf{m})$, $\mathbf{z} = g(\mathbf{n}) = g \circ f(\mathbf{m})$. We also denote $L_{\mathbf{m}}(\mathbf{z}) = L(\mathbf{m}, \mathbf{z})$.

Recall that when for a function $L(\mathbf{u}, \mathbf{v})$ and functions $g(\mathbf{t})$, $h(\mathbf{t})$, the composed partial derivatives of $L(g(\mathbf{t}), h(\mathbf{t}))$ are¹ :

$$\frac{\partial L}{\partial \mathbf{t}_k}(g(\mathbf{t}), h(\mathbf{t})) = \sum_i \frac{\partial L}{\partial \mathbf{u}_i}(g(\mathbf{t})) \times \frac{\partial g_i}{\partial \mathbf{t}_k}(\mathbf{t}) + \sum_j \frac{\partial L}{\partial \mathbf{v}_j}(h(\mathbf{t})) \times \frac{\partial h_j}{\partial \mathbf{t}_k}(\mathbf{t})$$

We assume we know $\frac{\partial L}{\partial \mathbf{u}_k}$ and $\frac{\partial L}{\partial \mathbf{v}_k}$. Using the chain rule for composed functions, we have :

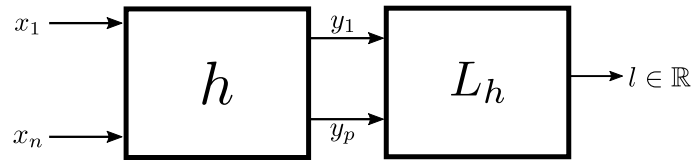
$$\begin{aligned} \frac{\partial L_f}{\partial \mathbf{m}_k}(\mathbf{m}) &= \frac{\partial L}{\partial \mathbf{m}_k}(\mathbf{m}, \mathbf{z}) + \sum_i \frac{\partial L_{\mathbf{m}} \circ g}{\partial \mathbf{n}_i}(\mathbf{n}) \times \frac{\partial f_i}{\partial \mathbf{m}_k}(\mathbf{m}) \\ \frac{\partial L_{\mathbf{m}} \circ g}{\partial \mathbf{n}_k}(\mathbf{n}) &= \sum_i \frac{\partial L_m}{\partial \mathbf{z}_i}(\mathbf{z}) \times \frac{\partial g_i}{\partial \mathbf{n}_k}(\mathbf{n}) = \sum_i \frac{\partial L}{\partial \mathbf{z}_i}(\mathbf{m}, \mathbf{z}) \times \frac{\partial g_i}{\partial \mathbf{n}_k}(\mathbf{n}) \end{aligned}$$

1. $\frac{\partial L(\mathbf{u}, \mathbf{v})}{\partial \mathbf{u}_k}$ is just a notation to indicate that we take the partial derivative with respect to the k -th dimension of the first variable of L

Looking at the equations above, we observe that to compute the partial derivatives with respect to all inputs (in green), it suffices to propagate information from child functions to their parents and to know how to compute the partial derivatives of functions with respect to their inputs (the derivatives marked in red that can be computed analytically).

In particular, for the function g , it suffices to know $\frac{\partial L_m}{\partial \mathbf{z}_i}$ (transmitted by the function L) and the analytical form of the partial derivatives $\frac{\partial g_i}{\partial \mathbf{n}_k}$ of g with respect to one of its inputs. **Autograd** (in PyTorch) allows doing this automatically by recording the parent functions for each computation performed.

2 Function in a Graph



Suppose we have a function $h : \mathbb{R}^n \mapsto \mathbb{R}^p$ and its output is used to compute an objective function L . The inputs $\mathbf{x}_1, \dots, \mathbf{x}_n$ can come from other functions – but this doesn't matter. The output of the function h is a vector $h(\mathbf{x}) = \mathbf{y} \in \mathbb{R}^p$.

Question 1

Calculate the partial derivative

$$\frac{\partial L \circ h}{\partial \mathbf{x}_i}(\mathbf{x})$$

as a function of $\frac{\partial h_j}{\partial \mathbf{x}_i}(\mathbf{x})$ and $(\nabla L)_j = \frac{\partial L}{\partial \mathbf{y}_j}(h(\mathbf{x}))$ where $(\nabla L)_j$ simplifies the notation. For a known function h (which is always the case for us), we can explicitly compute its derivative with respect to $\mathbf{x}_i$.

The previous calculation allows intuiting a result we will revisit in the following practical sessions : to compute the gradient of a composed function analytically, it suffices to know the gradient of each function with respect to each of its inputs and apply the chain rule.

Modern deep learning platforms exploit this result to modularize and make programming new architectures very efficient. For this, a *function* must implement not only the result of its application - called the *forward* pass - but also the partial derivative with respect to each of its inputs - called the *backward* pass. In particular,

Forward Computation of $y = h(\mathbf{x})$ as a function of $\mathbf{x}$

Backward Computation of $\frac{\partial h}{\partial \mathbf{x}}(\mathbf{x})$ and return taking into account the $\nabla L(\mathbf{y})$ passed as parameters and data derived from the forward phase ($\mathbf{x}$). Some partial computations can also be saved to accelerate calculations.

3 Application : Linear Regression

3.1 Gradient Computation (Scalar)

The linear regression model can be seen as the composition of two elements :

- a linear function, responsible for computing the prediction :

$$\hat{y} = f(\mathbf{x}, \mathbf{w}, b) = \mathbf{x} \cdot \mathbf{w} + b$$

- a mean squared error cost (MSE) : $mse(\hat{y}, y) = (\hat{y} - y)^2$

To perform gradient descent, the gradient of the cost must be computed with respect to the parameters $\mathbf{w}$ and b . We will compute this gradient through chaining of partial derivatives. To handle the general case, we assume the cost is defined as follows :

$$C(\mathbf{x}, \mathbf{w}, b, y) = mse(f(\mathbf{x}, \mathbf{w}, b), y)$$

where $\mathbf{x}$ is the input, y the desired output in $\mathbb{R}$, and $\mathbf{w} \in \mathbb{R}^n$ and $b \in \mathbb{R}$ the parameters of the linear model.

To compute the gradient of C with respect to $\mathbf{w}$ and b , which allows optimizing the parameters via gradient descent, we must first examine the gradients of the functions f and mse .

Question 2

Apply the result found in question 1 to compute

- The derivatives for the MSE cost function

$$\frac{\partial L \circ mse}{\partial y}(\hat{y}, y) \text{ and } \frac{\partial L \circ mse}{\partial \hat{y}}(\hat{y}, y)$$

- The derivatives of the linear function f with respect to its inputs $\mathbf{x}$ ($\mathbf{w}$ and b are obtained similarly)

$$\frac{\partial L \circ f}{\partial \mathbf{x}_j}(\mathbf{x}, \mathbf{w}, b)$$

The function L introduced here is generic and is present only to properly show how derivative chaining works. Regarding the problem that interests us, in the first case since the mse function is the last computation step, we can consider that L corresponds to the identity ; in the second case it is of course the function $mse(\cdot, \hat{y})$.

3.2 Gradient Computation (Matrix)

We now consider a more general case, which will be the one used in PyTorch, where we will work with a batch of q examples (instead of one) and p outputs (instead of one).

- We still have a linear function, responsible for computing the prediction :

$$\hat{\mathbf{Y}} = f(\mathbf{X}, \mathbf{W}, \mathbf{b}) = \mathbf{X}\mathbf{W} + \begin{pmatrix} \mathbf{b} \\ \vdots \\ \mathbf{b} \end{pmatrix}$$

where $\mathbf{X} \in \mathbb{R}^{q \times n}$ are the inputs, $\mathbf{W} \in \mathbb{R}^{n \times p}$ is the weight matrix, and $\mathbf{b} \in \mathbb{R}^{1 \times p}$ is the bias². The output $\mathbf{Y}$ is therefore a matrix in $\mathbb{R}^{q \times p}$.

- a mean squared error cost (MSE) generalized to a batch of examples and multiple outputs : $mse(\hat{\mathbf{Y}}, \mathbf{Y}) = \frac{1}{q} \|\hat{\mathbf{Y}} - \mathbf{Y}\|^2$. Note that we divide by the number of examples, which allows having a gradient magnitude that does not vary with the number of examples.

The cost C is then defined as

$$C(\mathbf{X}, \mathbf{W}, \mathbf{b}, \mathbf{Y}) = mse(f(\mathbf{X}, \mathbf{W}, \mathbf{b}), \mathbf{Y})$$

Question 3

Extend the results obtained in the previous question to compute

- The derivatives for the MSE cost function

$$\frac{\partial L \circ mse}{\partial \mathbf{Y}_{ij}}(\hat{\mathbf{Y}}, \mathbf{Y})$$

and

$$\frac{\partial L \circ mse}{\partial \hat{\mathbf{Y}}_{ij}}(\hat{\mathbf{Y}}, \mathbf{Y})$$

- The derivatives of the linear function f with respect to its inputs

$$\frac{\partial L \circ f}{\partial \mathbf{X}_{ij}}(\mathbf{X}, \mathbf{W}, \mathbf{b}), \frac{\partial L \circ f}{\partial \mathbf{W}_{ij}}(\mathbf{X}, \mathbf{W}, \mathbf{b}) \text{ and } \frac{\partial L \circ f}{\partial \mathbf{b}_i}(\mathbf{X}, \mathbf{W}, \mathbf{b})$$

To optimize computations and best exploit parallel architectures (GPU), we must then try to express calculations in matrix form as much as possible.

2. With PyTorch tensors (similar to numpy tensors), this is written $\mathbf{X} @ \mathbf{W} + \mathbf{b}$ using matrix multiplication and *broadcast*

Question 4

Use matrix notation to express the previous results, i.e. compute

- The derivatives for the MSE cost function

$$\frac{\partial L \circ mse}{\partial \mathbf{Y}}(\hat{\mathbf{Y}}, \mathbf{Y})$$

and

$$\frac{\partial L \circ mse}{\partial \hat{\mathbf{Y}}}(\hat{\mathbf{Y}}, \mathbf{Y})$$

- The derivatives of the linear function f with respect to its inputs

$$\frac{\partial L \circ f}{\partial \mathbf{X}}(\mathbf{X}, \mathbf{W}, \mathbf{b}), \frac{\partial L \circ f}{\partial \mathbf{W}}(\mathbf{X}, \mathbf{W}, \mathbf{b}) \text{ and } \frac{\partial L \circ f}{\partial \mathbf{b}}(\mathbf{X}, \mathbf{W}, \mathbf{b})$$

3.3 Gradient Computation of C

Now that we have computed all the partial derivatives, we must show how to compute the derivative of C with respect to each of its inputs.

Question 5

Give the formulas that allow computing

$$\frac{\partial C}{\partial \mathbf{W}} \text{ and } \frac{\partial C}{\partial \mathbf{b}}$$

You can use the function $mse_{\mathbf{Y}}(\hat{\mathbf{Y}}) = mse(\hat{\mathbf{Y}}, \mathbf{Y})$.

4 Function Implementation

PyTorch uses an abstract class `Function` from which all functions inherit and which requires implementing these two methods :

- method `forward(ctx, *inputs)` : computes the result of applying the function
- method `backward(ctx, *grad_outputs)` : computes the partial gradient with respect to each input of the `forward` method ; the number of `grad_outputs` must equal the number of outputs of `forward` (why?) and the number of outputs must equal the number of `inputs` of `forward`.

For implementation reasons, both methods must be static. The first parameter `ctx` allows saving a context during the `forward` pass (for example input tensors) and it is passed during

the **backward** pass as a parameter to retrieve the values. **Warning** : the context must be unique for each call to **forward**.

Question 6

Download the code provided on the course website. It contains a skeleton for implementing functions, a **Context** class to make your code work and an example of use.

1. Implement in **PyTorch** the necessary classes for linear regression : the linear function and the MSE cost function. Use proper **pyTorch** tools, particularly **Tensor** and not **numpy** matrices. Make sure your functions take as input batches of examples (2D matrix) and not a single example (vector). Don't hesitate to take an example and determine the dimensions of the different matrices involved.
2. **PyTorch** allows you to verify the computation of your derivatives using the function **gradcheck** which performs a numerical approximation by finite difference. An example of use is given in the source code. Test your functions with this tool, using **tp1_gradcheck.py** which you must complete.

Gradient Descent

The function f corresponds to the predictor (or classifier) that we want to learn, i.e. find the parameters $\mathbf{W}$ and $\mathbf{b}$ that minimize the error risk on predictions. For each set of examples $\mathbf{X}$ given, the output of the predictor $\hat{\mathbf{Y}}$ is compared to the expected output $\mathbf{Y}$ using the cost function $L(\hat{\mathbf{y}}, \mathbf{y})$ which quantifies the error. For parameters $\mathbf{W}$ and $\mathbf{b}$, the cost associated with the training set is

$$C(\mathbf{W}, \mathbf{b}) = mse(f(\mathbf{X}, \mathbf{W}, \mathbf{b}), \mathbf{Y})$$

In the context of empirical risk minimization, the formalization of the learning problem is as follows : find the optimal parameter $\mathbf{w}^*$ that minimizes the prediction cost on the training dataset

$$\mathbf{W}^*, \mathbf{b}^* = argmin_{\mathbf{w}, \mathbf{b}} L(\mathbf{W}, \mathbf{b}) = argmin_{\mathbf{W}, \mathbf{b}} mse(f(\mathbf{X}, \mathbf{W}, \mathbf{b}), \mathbf{Y})$$

A classical learning algorithm to optimize the parameter is the gradient descent algorithm. It consists of iteratively updating the parameter $\mathbf{w}$ according to the formula :

$$\mathbf{w} \leftarrow \mathbf{w} - \epsilon \nabla_{\mathbf{w}} L(\mathbf{W}, \mathbf{b})$$

$$\mathbf{b} \leftarrow \mathbf{b} - \epsilon \nabla_{\mathbf{b}} L(\mathbf{W}, \mathbf{b})$$

with ϵ a parameter called the learning rate.

Question 7

1. Implement a batch gradient descent algorithm for linear regression using automatic differentiation functionalities.
2. Test your implementation with the California Housing data. Plot the cost curve for training and testing. Preferably use `tensorboard` for this. For now, only use the `add_scalar` function after creating a log file using the command `SummaryWriter(path)`.
3. Implement gradient descent starting from the file `tp1_descente.py`.
4. Implement stochastic gradient descent and mini-batch gradient descent. Compare the convergence speed and the results obtained.